

Tarea 7

Estructura de datos de diapositivas (Lista Ligada)

TDA Lista Ligada genérica probada con la clase **PolloAsado**

Materia	Estructura de Datos
Carrera	Ingeniería en Computación - 3er semestre (FES Aragón)
Lenguaje	Java 21 (IntelliJ IDEA)
Repositorio	GitHub - Estructura de datos / Tarea 7
Contenido	Código fuente, explicación, pruebas y evidencias de ejecución

1. Objetivo

Terminar de implementar la estructura de datos **Lista Ligada** vista en las diapositivas, agregando los métodos que faltaban, y comprobar que funciona con una clase distinta de `String`. Para ello se utilizó la clase **PolloAsado** (corte, salsa y precio). Como la lista se declara con genéricos (`ListaLigadaADT<T>`), puede almacenar cualquier objeto sin modificar su código.

2. Descripción de la estructura

Una lista ligada simple es una colección de **nodos** enlazados en un solo sentido. Cada nodo guarda un dato y una referencia al nodo siguiente; el último apunta a `null`. La lista solo conoce el primer nodo (`head`), por lo que para llegar a cualquier posición hay que recorrerla desde el inicio.

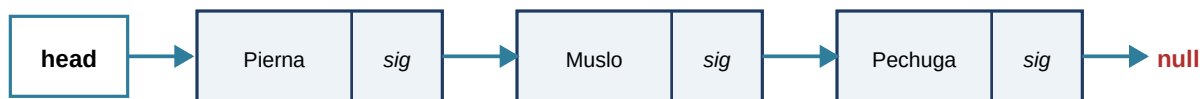


Figura 1. Lista ligada con tres nodos de tipo `PolloAsado` (se muestra solo el corte).

Clases del proyecto

Clase	Función
<code>Nodo<T></code>	Nodo genérico: atributo dato y referencia siguiente, con getters y setters.
<code>ListaLigadaADT<T></code>	TDA lista ligada. Mantiene head y contiene los métodos de las diapositivas y los nuevos.
<code>PolloAsado</code>	Clase de prueba (corte, salsa, precio) con <code>equals()</code> , <code>hashCode()</code> y <code>toString()</code> .
<code>Main</code>	Demostración paso a paso de todos los métodos.
<code>PruebasListaLigada</code>	34 pruebas automáticas que comparan resultado esperado contra obtenido (PASS/FAIL).

Por qué `PolloAsado` sobrescribe `equals()`

Los métodos `actualizar`, `agregarDespuesDe`, `eliminar`, `contiene` e `indiceDe` buscan datos con `equals()`. En `String` ya está implementado, pero en una clase propia, si no se sobrescribe, `equals()` compara referencias de memoria y dos pollos con los mismos valores se considerarían distintos. Por eso **PolloAsado** compara corte, salsa y precio, y se sobrescribe también `hashCode()` para mantener el contrato entre ambos.

3. Métodos de la lista

Los primeros seis venían en las diapositivas; los marcados como **Nuevo** son los que se implementaron en esta tarea. La complejidad se expresa en función de n , el número de nodos.

Método	Origen	Descripción	Complejidad
agregar(T)	Diapositivas	Agrega al final de la lista.	O(n)
transversal()	Diapositivas	Imprime todos los datos ("Vacía" si no hay).	O(n)
actualizar(T, T)	Diapositivas (corregido)	Reemplaza el primer dato igual al buscado.	O(n)
getTamaño()	Diapositivas	Cuenta los nodos.	O(n)
agregarDespuesDe(T, T)	Diapositivas (corregido)	Inserta después de la referencia.	O(n)
estaVacía()	Nuevo	true si head es null.	O(1)
agregarAlInicio(T)	Nuevo	Inserta un nuevo head.	O(1)
agregarAntesDe(T, T)	Nuevo	Inserta antes de la referencia; false si no existe.	O(n)
insertarEn(int, T)	Nuevo	Inserta en una posición (0 a tamaño); false si es inválida.	O(n)
contiene(T)	Nuevo	Indica si el dato existe.	O(n)
indiceDe(T)	Nuevo	Posición del dato, o -1.	O(n)
obtener(int)	Nuevo	Dato en una posición, o null si está fuera de rango.	O(n)
eliminar(T)	Nuevo	Elimina el primer nodo igual; false si no existe.	O(n)
eliminarPrimero()	Nuevo	Quita y regresa el primer dato (null si vacía).	O(1)
eliminarUltimo()	Nuevo	Quita y regresa el último dato (null si vacía).	O(n)
vaciar()	Nuevo	Elimina todos los nodos.	O(1)
toString()	Nuevo	Cadena con el formato dato dato .	O(n)

Correcciones al código de las diapositivas

Método	Problema detectado	Corrección
actualizar()	Si el dato no existía, el ciclo recorría hasta null y lanzaba NullPointerException.	Se busca con un método auxiliar y se avisa "No se encontro".
agregarDespuesDe()	Mismo problema cuando la referencia no existía.	Misma verificación antes de insertar.
transversal()	No imprimía salto de línea al final.	Se agregó el cierre " " y el salto de línea.

4. Código fuente

Nodo.java

```
package listaligada;

/**
 * Nodo genérico de una lista ligada simple.
 * Guarda un dato de tipo T y la referencia al siguiente nodo.
 */
public class Nodo<T> {
    private T dato;
    private Nodo<T> siguiente;

    public Nodo(T dato) {
        this.dato = dato;
        this.siguiente = null;
    }

    public Nodo(T dato, Nodo<T> siguiente) {
        this.dato = dato;
        this.siguiente = siguiente;
    }

    public T getDato() {
        return dato;
    }

    public void setDato(T dato) {
        this.dato = dato;
    }

    public Nodo<T> getSiguiente() {
        return siguiente;
    }

    public void setSiguiente(Nodo<T> siguiente) {
        this.siguiente = siguiente;
    }
}
```

PolloAsado.java

```
package listaligada;

import java.util.Locale;
import java.util.Objects;

/**
 * Clase de prueba (en lugar de String) para almacenar en la lista ligada.
 * Representa una orden de pollo asado.
 *
 * IMPORTANTE: se sobreescribe equals() porque la lista compara datos con
 * equals() en actualizar(), agregarDespuesDe(), eliminar(), contiene(), etc.
 * Sin equals() se compararían referencias de memoria y la búsqueda fallaría.
 */
public class PolloAsado {
    private String corte;    // Pierna, Muslo, Pechuga, Alas, Entero...
    private String salsa;    // BBQ, Adobada, Chimichurri...
    private double precio;

    public PolloAsado(String corte, String salsa, double precio) {
        this.corte = corte;
        this.salsa = salsa;
        this.precio = precio;
    }

    public String getCorte() { return corte; }
    public String getSalsa() { return salsa; }
    public double getPrecio() { return precio; }

    public void setCorte(String corte) { this.corte = corte; }
    public void setSalsa(String salsa) { this.salsa = salsa; }
    public void setPrecio(double precio) { this.precio = precio; }
}
```

```

@Override
public boolean equals(Object o) {
    if (this == o) return true;
    if (o == null || getClass() != o.getClass()) return false;
    PolloAsado otro = (PolloAsado) o;
    return Double.compare(precio, otro.precio) == 0
        && Objects.equals(corte, otro.corte)
        && Objects.equals(salsa, otro.salsa);
}

@Override
public int hashCode() {
    return Objects.hash(corte, salsa, precio);
}

@Override
public String toString() {
    return String.format(Locale.US, "%s (%s) $%.2f", corte, salsa, precio);
}
}

```

ListaLigadaADT.java

```

package listaligada;

/**
 * TDA Lista Ligada simple (genérica).
 *
 * Métodos que venían en las diapositivas: constructor, agregar, transversal,
 * actualizar, getTamaño y agregarDespuesDe.
 * Métodos implementados en esta tarea: estaVacía, agregarAlInicio,
 * agregarAntesDe, insertarEn, contiene, indiceDe, obtener, eliminar,
 * eliminarPrimero, eliminarUltimo, vaciar y toString.
 *
 * @param <T> tipo de dato almacenado (ya no solo String).
 */
public class ListaLigadaADT<T> {

    private Nodo<T> head;

    public ListaLigadaADT() {
        this.head = null;
    }

    // -----
    // MÉTODOS DE LAS DIAPOSITIVAS
    // -----

    /** Agrega un dato al final de la lista. */
    public void agregar(T dato) {
        if (head == null) {
            this.head = new Nodo<>(dato);
        } else {
            Nodo<T> actual = head;
            while (actual.getSiguiente() != null) {
                actual = actual.getSiguiente();
            }
            actual.setSiguiente(new Nodo<>(dato));
        }
    }

    /** Recorre la lista imprimiendo cada dato. Se agrega salto de línea final. */
    public void transversal() {
        if (head == null) {
            System.out.println("Vacía");
        } else {
            Nodo<T> actual = head;
            do {
                System.out.print("|" + actual.getDato());
                actual = actual.getSiguiente();
            } while (actual != null);
            System.out.println("|");
        }
    }
}

```

```

* Reemplaza el primer dato igual a aBuscar por nuevoValor.
* (Corrección: el código original lanzaba NullPointerException si el
* dato no existía; ahora avisa que no se encontró.)
*/
public void actualizar(T aBuscar, T nuevoValor) {
    if (head == null) {
        System.out.println("Vacía");
    } else {
        Nodo<T> actual = buscarNodo(aBuscar);
        if (actual == null) {
            System.out.println("No se encontro: " + aBuscar);
        } else {
            actual.setDato(nuevoValor);
        }
    }
}

/** Regresa la cantidad de nodos de la lista. */
public int getTamanio() {
    int contador = 0;
    if (head == null) {
        return contador;
    } else {
        Nodo<T> actual = head;
        do {
            contador++;
            actual = actual.getSiguiente();
        } while (actual != null);
        return contador;
    }
}

/**
* Inserta valor justo después del primer nodo igual a referencia.
* (Corrección: se evita NullPointerException si la referencia no existe.)
*/
public void agregarDespuesDe(T referencia, T valor) {
    if (head == null) {
        System.out.println("Vacía");
    } else {
        Nodo<T> actual = buscarNodo(referencia);
        if (actual == null) {
            System.out.println("No se encontro la referencia: " + referencia);
        } else {
            Nodo<T> nuevoNodo = new Nodo<>(valor, actual.getSiguiente());
            actual.setSiguiente(nuevoNodo);
        }
    }
}

// -----
// MÉTODOS FALTANTES IMPLEMENTADOS
// -----

/** @return true si la lista no tiene nodos. */
public boolean estaVacía() {
    return head == null;
}

/** Agrega un dato al inicio de la lista (nuevo head). */
public void agregarAlInicio(T dato) {
    head = new Nodo<>(dato, head);
}

/**
* Inserta valor justo antes del primer nodo igual a referencia.
* @return true si se insertó, false si la referencia no existe.
*/
public boolean agregarAntesDe(T referencia, T valor) {
    if (head == null) {
        return false;
    }
    if (head.getDato().equals(referencia)) {
        agregarAlInicio(valor);
        return true;
    }
    Nodo<T> anterior = head;

```

```

        while (anterior.getSiguiente() != null
            && !anterior.getSiguiente().getDato().equals(referencia)) {
            anterior = anterior.getSiguiente();
        }
        if (anterior.getSiguiente() == null) {
            return false;
        }
        anterior.setSiguiente(new Nodo<>(valor, anterior.getSiguiente()));
        return true;
    }

    /**
     * Inserta un dato en la posición indicada (0 = inicio, tamaño = final).
     * @return true si el índice era válido.
     */
    public boolean insertarEn(int indice, T dato) {
        if (indice < 0 || indice > getTamaño()) {
            return false;
        }
        if (indice == 0) {
            agregarAlInicio(dato);
            return true;
        }
        Nodo<T> anterior = head;
        for (int i = 0; i < indice - 1; i++) {
            anterior = anterior.getSiguiente();
        }
        anterior.setSiguiente(new Nodo<>(dato, anterior.getSiguiente()));
        return true;
    }

    /** @return true si existe un dato igual a buscado. */
    public boolean contiene(T buscado) {
        return buscarNodo(buscado) != null;
    }

    /** @return posición (desde 0) del primer dato igual a buscado, o -1. */
    public int indiceDe(T buscado) {
        int indice = 0;
        Nodo<T> actual = head;
        while (actual != null) {
            if (actual.getDato().equals(buscado)) {
                return indice;
            }
            actual = actual.getSiguiente();
            indice++;
        }
        return -1;
    }

    /** @return dato en la posición indicada, o null si el índice es inválido. */
    public T obtener(int indice) {
        if (indice < 0) {
            return null;
        }
        Nodo<T> actual = head;
        int i = 0;
        while (actual != null && i < indice) {
            actual = actual.getSiguiente();
            i++;
        }
        return (actual == null) ? null : actual.getDato();
    }

    /**
     * Elimina el primer nodo cuyo dato sea igual a aEliminar.
     * @return true si se eliminó, false si no existía.
     */
    public boolean eliminar(T aEliminar) {
        if (head == null) {
            return false;
        }
        if (head.getDato().equals(aEliminar)) {
            head = head.getSiguiente();
            return true;
        }
        Nodo<T> anterior = head;

```

```

        while (anterior.getSiguiente() != null
            && !anterior.getSiguiente().getDato().equals(aEliminar)) {
            anterior = anterior.getSiguiente();
        }
        if (anterior.getSiguiente() == null) {
            return false;
        }
        anterior.setSiguiente(anterior.getSiguiente().getSiguiente());
        return true;
    }

    /** Elimina el primer nodo. @return el dato eliminado o null si estaba vacía. */
    public T eliminarPrimero() {
        if (head == null) {
            return null;
        }
        T dato = head.getDato();
        head = head.getSiguiente();
        return dato;
    }

    /** Elimina el último nodo. @return el dato eliminado o null si estaba vacía. */
    public T eliminarUltimo() {
        if (head == null) {
            return null;
        }
        if (head.getSiguiente() == null) {
            T dato = head.getDato();
            head = null;
            return dato;
        }
        Nodo<T> anterior = head;
        while (anterior.getSiguiente().getSiguiente() != null) {
            anterior = anterior.getSiguiente();
        }
        T dato = anterior.getSiguiente().getDato();
        anterior.setSiguiente(null);
        return dato;
    }

    /** Elimina todos los nodos. */
    public void vaciar() {
        head = null;
    }

    @Override
    public String toString() {
        if (head == null) {
            return "Vacía";
        }
        StringBuilder sb = new StringBuilder();
        Nodo<T> actual = head;
        while (actual != null) {
            sb.append("|").append(actual.getDato());
            actual = actual.getSiguiente();
        }
        return sb.append("|").toString();
    }

    // -----
    // AUXILIAR PRIVADO
    // -----

    /** Regresa el primer nodo cuyo dato sea igual a buscado, o null. */
    private Nodo<T> buscarNodo(T buscado) {
        Nodo<T> actual = head;
        while (actual != null && !actual.getDato().equals(buscado)) {
            actual = actual.getSiguiente();
        }
        return actual;
    }
}

```

Main.java

```
package listaligada;
```

```

/**
 * Demostración de la Lista Ligada genérica usando la clase PolloAsado
 * (en lugar de String). Recorre todos los métodos del TDA.
 */
public class Main {

    private static void titulo(String texto) {
        System.out.println();
        System.out.println("=== " + texto + " ===");
    }

    /** Muestra la lista un elemento por renglon (mas legible que transversal). */
    private static void mostrar(ListaLigadaADT<PolloAsado> lista) {
        if (lista.estaVacia()) {
            System.out.println(" (lista vacia)");
            return;
        }
        for (int i = 0; i < lista.getTamanio(); i++) {
            System.out.println(" [" + i + "] " + lista.obtener(i));
        }
    }

    public static void main(String[] args) {
        ListaLigadaADT<PolloAsado> menu = new ListaLigadaADT<>();

        PolloAsado pierna = new PolloAsado("Pierna", "BBQ", 85.00);
        PolloAsado muslo = new PolloAsado("Muslo", "Adobada", 70.00);
        PolloAsado pechuga = new PolloAsado("Pechuga", "Chimichurri", 95.50);
        PolloAsado alas = new PolloAsado("Alas", "Picante", 60.00);
        PolloAsado entero = new PolloAsado("Entero", "Limon", 210.00);

        titulo("1. Lista recién creada");
        System.out.println("Esta vacia? " + menu.estaVacia());
        System.out.print("Transversal: ");
        menu.transversal();
        System.out.println("Tamanio: " + menu.getTamanio());

        titulo("2. agregar() al final");
        menu.agregar(pierna);
        menu.agregar(muslo);
        menu.agregar(pechuga);
        System.out.print("Transversal: ");
        menu.transversal();
        System.out.println("Tamanio: " + menu.getTamanio());

        titulo("3. agregarAlInicio()");
        menu.agregarAlInicio(alas);
        mostrar(menu);

        titulo("4. agregarDespuesDe(Muslo, Entero)");
        menu.agregarDespuesDe(muslo, entero);
        mostrar(menu);

        titulo("5. agregarAntesDe(Pechuga, Pierna BBQ grande)");
        PolloAsado piernaGrande = new PolloAsado("Pierna", "BBQ grande", 110.00);
        System.out.println("Insertado? " + menu.agregarAntesDe(pechuga, piernaGrande));
        mostrar(menu);

        titulo("6. insertarEn(2, Combo familiar)");
        PolloAsado combo = new PolloAsado("Combo familiar", "Mixta", 320.00);
        System.out.println("Insertado? " + menu.insertarEn(2, combo));
        System.out.println("insertarEn(99, ...) -> " + menu.insertarEn(99, combo) + " (indice invalido)");
        mostrar(menu);

        titulo("7. contiene() / indiceDe() / obtener()");
        System.out.println("contiene(Pechuga)? " + menu.contiene(pechuga));
        System.out.println("indiceDe(Pechuga) = " + menu.indiceDe(pechuga));
        System.out.println("obtener(0) = " + menu.obtener(0));
        System.out.println("obtener(50) = " + menu.obtener(50) + " (fuera de rango)");
        PolloAsado inexistente = new PolloAsado("Costilla", "Miel", 120.00);
        System.out.println("contiene(Costilla)? " + menu.contiene(inexistente));

        titulo("8. actualizar(Muslo $70.00 -> Muslo $55.00)");
        menu.actualizar(muslo, new PolloAsado("Muslo", "Adobada", 55.00));
        mostrar(menu);
        System.out.print("actualizar() de un dato inexistente: ");
        menu.actualizar(inexistente, pierna);
    }
}

```

```

        titulo("9. eliminar(Entero)");
        System.out.println("Eliminado? " + menu.eliminar(entero));
        System.out.println("Eliminar de nuevo? " + menu.eliminar(entero));
        mostrar(menu);

        titulo("10. eliminarPrimero() y eliminarUltimo()");
        System.out.println("Primero eliminado: " + menu.eliminarPrimero());
        System.out.println("Ultimo eliminado: " + menu.eliminarUltimo());
        mostrar(menu);
        System.out.println("Tamano: " + menu.getTamano());

        titulo("11. vaciar()");
        menu.vaciar();
        System.out.print("Transversal: ");
        menu.transversal();
        System.out.println("Esta vacia? " + menu.estaVacia());
        System.out.println("eliminarPrimero() en lista vacia = " + menu.eliminarPrimero());
        System.out.println("eliminarUltimo() en lista vacia = " + menu.eliminarUltimo());
    }
}

```

PruebasListaLigada.java

```

package listaligada;

/**
 * Pruebas automáticas (sin JUnit) de ListaLigadaADT<PolloAsado>.
 * Cada prueba compara el resultado obtenido contra el esperado e imprime PASS/FAIL.
 */
public class PruebasListaLigada {

    private static int total = 0;
    private static int fallidas = 0;

    private static void verificar(String nombre, Object esperado, Object obtenido) {
        total++;
        boolean ok = (esperado == null) ? obtenido == null : esperado.equals(obtenido);
        if (!ok) fallidas++;
        System.out.printf("[%s] %s%n", ok ? "PASS" : "FAIL", nombre);
        if (!ok) {
            System.out.println("    esperado: " + esperado);
            System.out.println("    obtenido: " + obtenido);
        }
    }

    public static void main(String[] args) {
        PolloAsado a = new PolloAsado("Pierna", "BBQ", 85.0);
        PolloAsado b = new PolloAsado("Muslo", "Adobada", 70.0);
        PolloAsado c = new PolloAsado("Pechuga", "Limon", 95.5);
        PolloAsado d = new PolloAsado("Alas", "Picante", 60.0);
        PolloAsado x = new PolloAsado("Entero", "Mixta", 210.0);

        ListaLigadaADT<PolloAsado> l = new ListaLigadaADT<>();

        verificar("Lista nueva: estaVacia()", true, l.estaVacia());
        verificar("Lista nueva: getTamano() == 0", 0, l.getTamano());
        verificar("Lista nueva: toString() == Vacia", "Vacía", l.toString());

        l.agregar(a); l.agregar(b); l.agregar(c);
        verificar("agregar() x3: tamano", 3, l.getTamano());
        verificar("agregar() x3: orden", "|" + a + "|" + b + "|" + c + "|", l.toString());

        l.agregarAlInicio(d);
        verificar("agregarAlInicio(): primer elemento", d, l.obtener(0));
        verificar("agregarAlInicio(): tamano", 4, l.getTamano());

        l.agregarDespuesDe(b, x);
        verificar("agregarDespuesDe(): posicion", 3, l.indiceDe(x));
        verificar("agregarDespuesDe(): tamano", 5, l.getTamano());

        verificar("agregarAntesDe(cabeza)", true, l.agregarAntesDe(d, a));
        verificar("agregarAntesDe(): nuevo head", a, l.obtener(0));
        verificar("agregarAntesDe(inexistente) == false", false,
            l.agregarAntesDe(new PolloAsado("X", "Y", 1), a));

        verificar("insertarEn(indice invalido) == false", false, l.insertarEn(-1, a));
    }
}

```

```

verificar("insertarEn(tamano) == true (al final)", true, l.insertarEn(l.getTamano(), x));
verificar("insertarEn(): ultimo elemento", x, l.obtener(l.getTamano() - 1));

verificar("contiene() existente", true, l.contiene(c));
verificar("contiene() inexistente", false, l.contiene(new PolloAsado("Z", "Z", 0)));
verificar("indiceDe() inexistente == -1", -1, l.indiceDe(new PolloAsado("Z", "Z", 0)));
verificar("obtener() fuera de rango == null", null, l.obtener(100));
verificar("obtener() negativo == null", null, l.obtener(-3));

PolloAsado nuevo = new PolloAsado("Muslo", "Adobada", 55.0);
l.actualizar(b, nuevo);
verificar("actualizar(): contiene nuevo", true, l.contiene(nuevo));
verificar("actualizar(): ya no contiene el viejo", false, l.contiene(b));

int antes = l.getTamano();
verificar("eliminar() existente", true, l.eliminar(c));
verificar("eliminar(): tamano - 1", antes - 1, l.getTamano());
verificar("eliminar() inexistente", false, l.eliminar(c));
verificar("eliminar(cabeza)", true, l.eliminar(a));

verificar("eliminarPrimero() regresa el dato", d, l.eliminarPrimero());
verificar("eliminarUltimo() regresa el dato", x, l.eliminarUltimo());

l.vaciar();
verificar("vaciar(): estaVacia()", true, l.estaVacia());
verificar("eliminarPrimero() en vacia == null", null, l.eliminarPrimero());
verificar("eliminarUltimo() en vacia == null", null, l.eliminarUltimo());
verificar("eliminar() en vacia == false", false, l.eliminar(a));

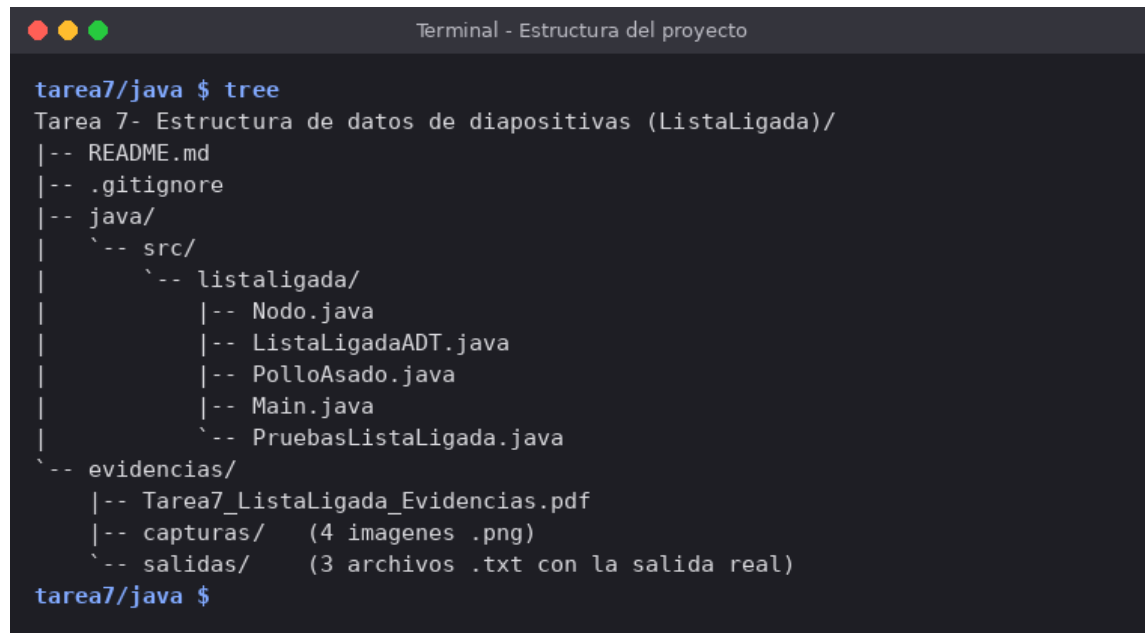
l.agregar(a);
verificar("eliminarUltimo() con 1 nodo regresa el dato", a, l.eliminarUltimo());
verificar("eliminarUltimo() con 1 nodo: queda vacia", true, l.estaVacia());

System.out.println();
System.out.printf("Resultado: %d/%d pruebas exitosas%n", total - fallidas, total);
if (fallidas > 0) System.exit(1);
}
}

```

5. Evidencias de ejecución

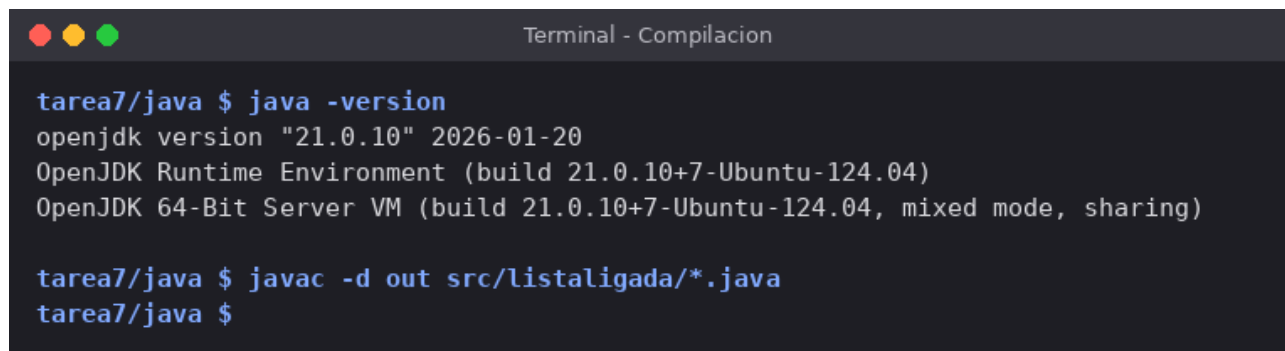
Las capturas corresponden a la compilación y ejecución reales del proyecto (Java 21). La salida completa en texto se guarda en la carpeta evidencias/salidas.



```
Terminal - Estructura del proyecto

tarea7/java $ tree
Tarea 7- Estructura de datos de diapositivas (ListaLigada)/
|-- README.md
|-- .gitignore
|-- java/
|   |-- src/
|   |   |-- listaligada/
|   |   |   |-- Nodo.java
|   |   |   |-- ListaLigadaADT.java
|   |   |   |-- PolloAsado.java
|   |   |   |-- Main.java
|   |   |-- PruebasListaLigada.java
|   |-- evidencias/
|   |   |-- Tarea7_ListaLigada_Evidencias.pdf
|   |   |-- capturas/      (4 imagenes .png)
|   |   |-- salidas/       (3 archivos .txt con la salida real)
tarea7/java $
```

Figura 2. Estructura de carpetas del proyecto.



```
Terminal - Compilacion

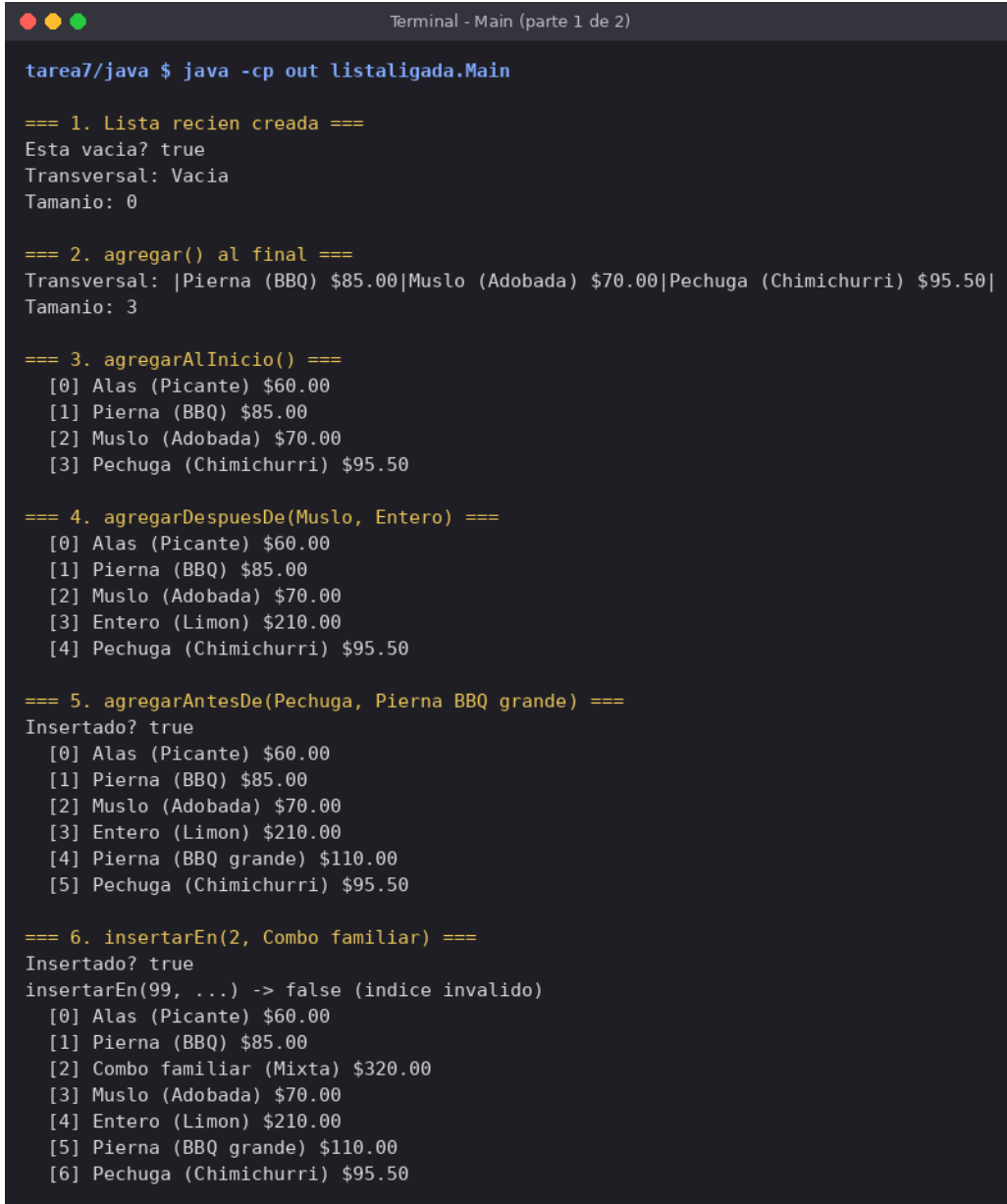
tarea7/java $ java -version
openjdk version "21.0.10" 2026-01-20
OpenJDK Runtime Environment (build 21.0.10+7-Ubuntu-124.04)
OpenJDK 64-Bit Server VM (build 21.0.10+7-Ubuntu-124.04, mixed mode, sharing)

tarea7/java $ javac -d out src/listaligada/*.java
tarea7/java $
```

Figura 3. Versión de Java y compilación sin errores ni advertencias.

5.1 Ejecución de Main (demostración)

Se prueba cada método con objetos **PolloAsado**: lista vacía, inserciones (final, inicio, antes, después y por índice), búsquedas, actualización, eliminaciones y casos límite (dato inexistente, índice inválido, lista vacía).



```
Terminal - Main (parte 1 de 2)

tarea7/java $ java -cp out listaligada.Main

=== 1. Lista recién creada ===
Esta vacia? true
Transversal: Vacía
Tamaño: 0

=== 2. agregar() al final ===
Transversal: |Pierna (BBQ) $85.00|Muslo (Adobada) $70.00|Pechuga (Chimichurri) $95.50|
Tamaño: 3

=== 3. agregarAlInicio() ===
[0] Alas (Picante) $60.00
[1] Pierna (BBQ) $85.00
[2] Muslo (Adobada) $70.00
[3] Pechuga (Chimichurri) $95.50

=== 4. agregarDespuesDe(Muslo, Entero) ===
[0] Alas (Picante) $60.00
[1] Pierna (BBQ) $85.00
[2] Muslo (Adobada) $70.00
[3] Entero (Limon) $210.00
[4] Pechuga (Chimichurri) $95.50

=== 5. agregarAntesDe(Pechuga, Pierna BBQ grande) ===
Insertado? true
[0] Alas (Picante) $60.00
[1] Pierna (BBQ) $85.00
[2] Muslo (Adobada) $70.00
[3] Entero (Limon) $210.00
[4] Pierna (BBQ grande) $110.00
[5] Pechuga (Chimichurri) $95.50

=== 6. insertarEn(2, Combo familiar) ===
Insertado? true
insertarEn(99, ...) -> false (índice inválido)
[0] Alas (Picante) $60.00
[1] Pierna (BBQ) $85.00
[2] Combo familiar (Mixta) $320.00
[3] Muslo (Adobada) $70.00
[4] Entero (Limon) $210.00
[5] Pierna (BBQ grande) $110.00
[6] Pechuga (Chimichurri) $95.50
```

Figura 4. Salida de Main, secciones 1 a 6.

```

=== 7. contiene() / indiceDe() / obtener() ===
contiene(Pechuga)? true
indiceDe(Pechuga) = 6
obtener(0) = Alas (Picante) $60.00
obtener(50) = null (fuera de rango)
contiene(Costilla)? false

=== 8. actualizar(Muslo $70.00 -> Muslo $55.00) ===
[0] Alas (Picante) $60.00
[1] Pierna (BBQ) $85.00
[2] Combo familiar (Mixta) $320.00
[3] Muslo (Adobada) $55.00
[4] Entero (Limon) $210.00
[5] Pierna (BBQ grande) $110.00
[6] Pechuga (Chimichurri) $95.50
actualizar() de un dato inexistente: No se encontro: Costilla (Miel) $120.00

=== 9. eliminar(Entero) ===
Eliminado? true
Eliminar de nuevo? false
[0] Alas (Picante) $60.00
[1] Pierna (BBQ) $85.00
[2] Combo familiar (Mixta) $320.00
[3] Muslo (Adobada) $55.00
[4] Pierna (BBQ grande) $110.00
[5] Pechuga (Chimichurri) $95.50

=== 10. eliminarPrimero() y eliminarUltimo() ===
Primero eliminado: Alas (Picante) $60.00
Ultimo eliminado: Pechuga (Chimichurri) $95.50
[0] Pierna (BBQ) $85.00
[1] Combo familiar (Mixta) $320.00
[2] Muslo (Adobada) $55.00
[3] Pierna (BBQ grande) $110.00
Tamano: 4

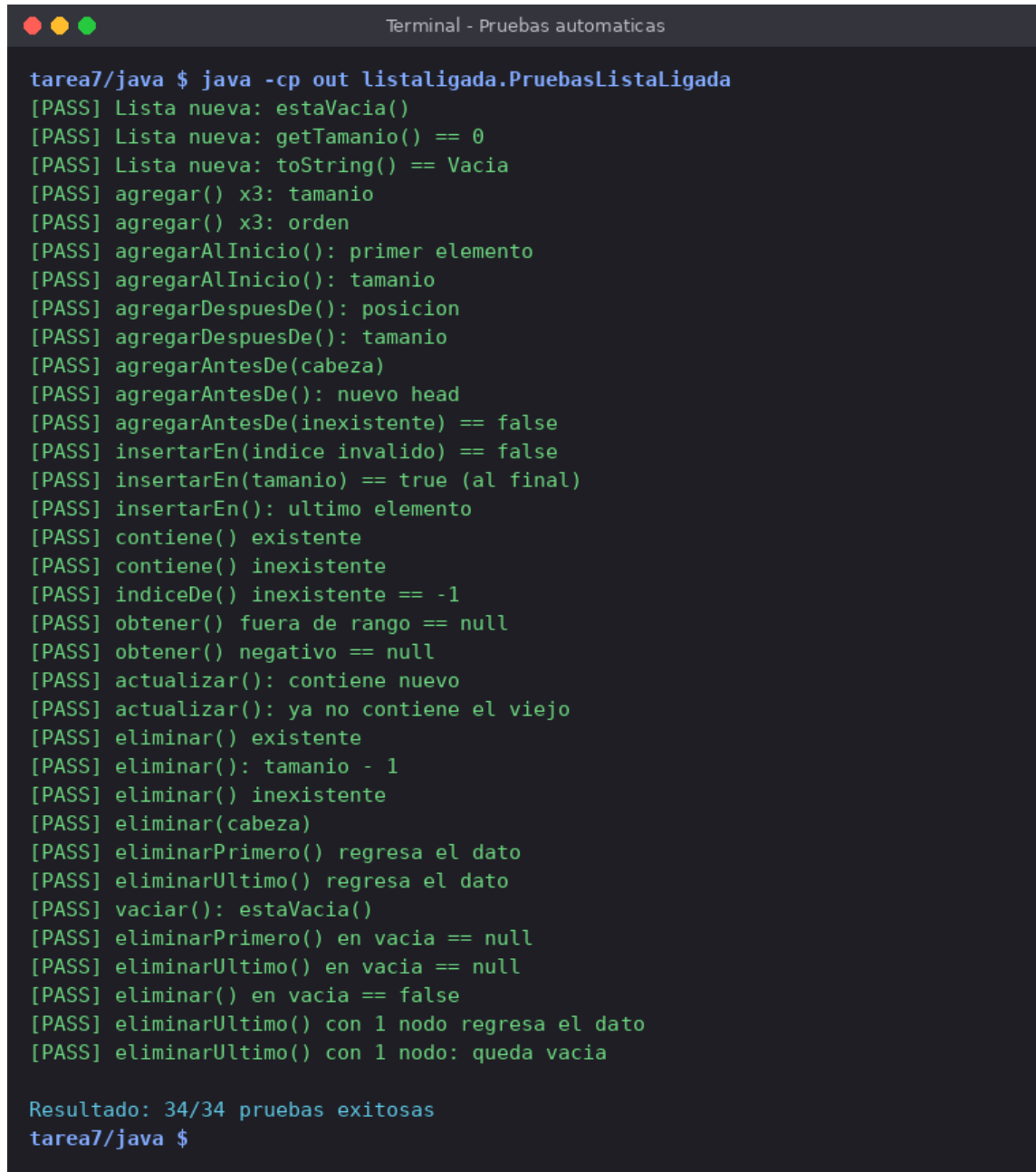
=== 11. vaciar() ===
Transversal: Vacia
Esta vacia? true
eliminarPrimero() en lista vacia = null
eliminarUltimo() en lista vacia = null
tarea7/java $

```

Figura 5. Salida de Main, secciones 7 a 11.

5.2 Pruebas automáticas

La clase **PruebasListaLigada** verifica 34 casos (incluyendo cabeza, último nodo, un solo nodo y lista vacía). Todas las pruebas resultaron exitosas.



```
Terminal - Pruebas automaticas

tarea7/java $ java -cp out listaligada.PruebasListaLigada
[PASS] Lista nueva: estaVacía()
[PASS] Lista nueva: getTamano() == 0
[PASS] Lista nueva: toString() == Vacía
[PASS] agregar() x3: tamano
[PASS] agregar() x3: orden
[PASS] agregarAlInicio(): primer elemento
[PASS] agregarAlInicio(): tamano
[PASS] agregarDespuesDe(): posicion
[PASS] agregarDespuesDe(): tamano
[PASS] agregarAntesDe(cabeza)
[PASS] agregarAntesDe(): nuevo head
[PASS] agregarAntesDe(inexistente) == false
[PASS] insertarEn(indice invalido) == false
[PASS] insertarEn(tamano) == true (al final)
[PASS] insertarEn(): ultimo elemento
[PASS] contiene() existente
[PASS] contiene() inexistente
[PASS] indiceDe() inexistente == -1
[PASS] obtener() fuera de rango == null
[PASS] obtener() negativo == null
[PASS] actualizar(): contiene nuevo
[PASS] actualizar(): ya no contiene el viejo
[PASS] eliminar() existente
[PASS] eliminar(): tamano - 1
[PASS] eliminar() inexistente
[PASS] eliminar(cabeza)
[PASS] eliminarPrimero() regresa el dato
[PASS] eliminarUltimo() regresa el dato
[PASS] vaciar(): estaVacía()
[PASS] eliminarPrimero() en vacía == null
[PASS] eliminarUltimo() en vacía == null
[PASS] eliminar() en vacía == false
[PASS] eliminarUltimo() con 1 nodo regresa el dato
[PASS] eliminarUltimo() con 1 nodo: queda vacía

Resultado: 34/34 pruebas exitosas
tarea7/java $
```

Figura 6. Resultado: 34/34 pruebas exitosas.

6. Conclusiones

La lista ligada quedó completa y genérica: al no depender de `String`, se probó con **PolloAsado** sin cambiar una sola línea de la estructura. El punto clave fue sobrescribir `equals()` en la clase almacenada, ya que todas las búsquedas dependen de él.

También se vio que, al no tener referencia al final (`tail`), agregar al final y eliminar el último cuestan $O(n)$, mientras que operar sobre el inicio es $O(1)$. Además, al extender código base conviene revisar los casos límite: los métodos de las diapositivas fallaban cuando el dato no existía y se corrigieron.

Posibles mejoras: agregar un atributo `tail` y un contador de tamaño para que `agregar()` y `getTamano()` sean $O(1)$, o implementar una lista doblemente ligada.